Q3: Hough Transform in Noisy Images

Course: Computer Vision (ITA05) | Assessment Tool 2: Case Study | CO3

Scenario

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results. Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Analysis

The Hough Transform detects lines by letting every edge pixel 'vote' for all possible lines that could pass through it in parameter (rho, theta) space. Peaks in this accumulator array correspond to lines present in the image. The technique therefore depends entirely on the quality of the edge map produced by the preceding edge-detection stage (usually Canny).

When an image contains noise (Gaussian sensor noise, salt-and-pepper impulse noise, or compression artifacts), the noise pixels themselves create false, disconnected intensity gradients. Canny edge detection reacts to these random gradients and marks thousands of spurious edge pixels that have nothing to do with real image structure.

Each of these spurious edge pixels casts votes into the Hough accumulator. Because noise is spread across the whole image, it raises the 'background' vote level everywhere, which (a) creates many false peaks that get reported as detected lines that do not really exist, and (b) can fragment a single true line into many short broken segments because noise breaks edge continuity along the line.

The experiment below confirms this: on a noisy synthetic image, a naive Canny + HoughLinesP pipeline reports over a thousand spurious line segments, essentially drowning out the four real lines in the scene.

Suggested Solution / Improvements

- Denoise before edge detection. A median filter is very effective against salt-and-pepper (impulse) noise because it replaces each pixel with the median of its neighbourhood, which is robust to outliers. A Gaussian blur afterwards suppresses remaining Gaussian sensor noise while preserving large-scale edges.
- Re-tune the Canny thresholds after denoising. Once the image is cleaner, the low/high hysteresis thresholds can be raised, so only strong, genuine gradients survive as edges instead of weak noise-driven gradients.
- Tune Hough parameters to demand more evidence per line. Raising the 'threshold' (minimum number of accumulator votes), 'minLineLength', and reducing 'maxLineGap' forces the algorithm to only report lines that are backed by a long, continuous run of edge pixels -- exactly the signature of a real structural line rather than noise.
- Morphological closing (dilate + erode) can also be applied to the edge map before Hough Transform to reconnect line segments that noise has fragmented, improving detection of long lines that noise has broken up.

Output / Result Screenshot

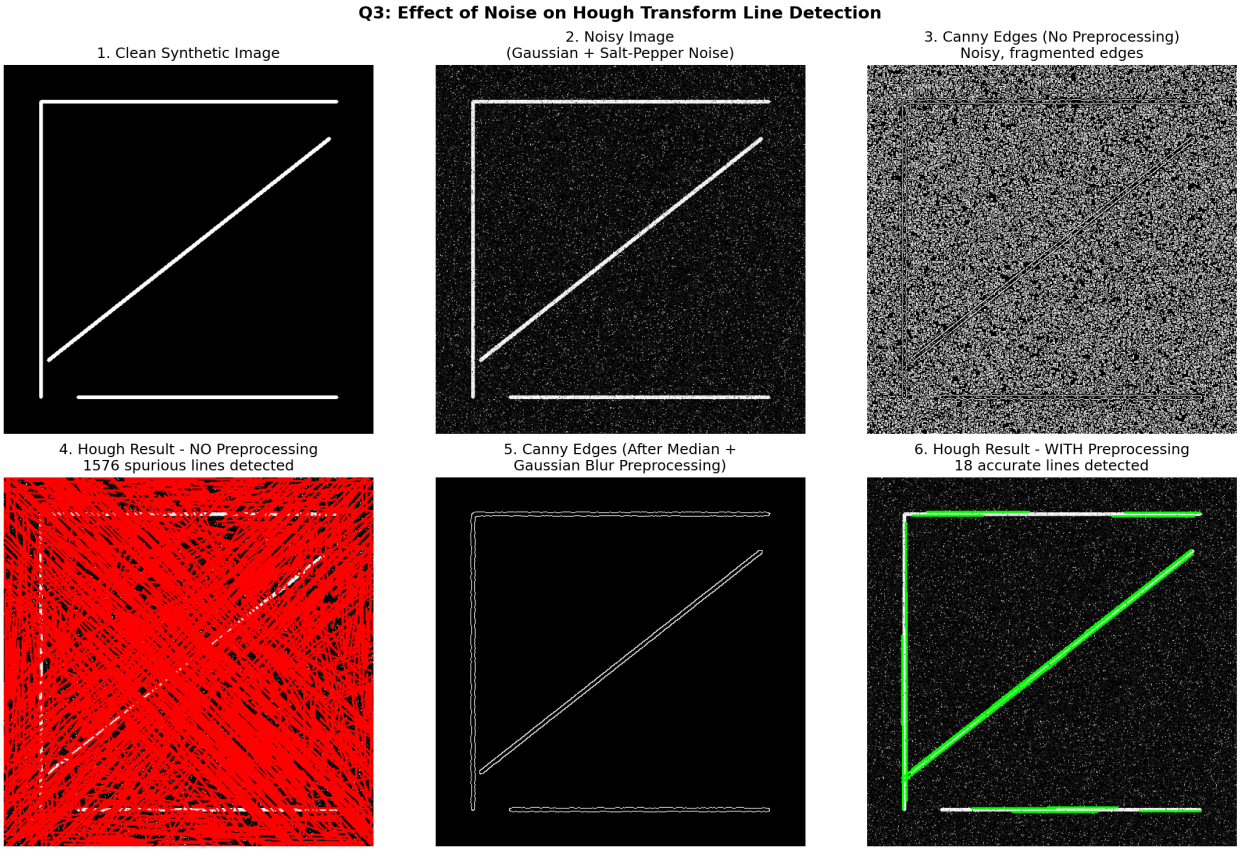

Figure: Output generated by Q3_hough_transform_noisy_images.py

Conclusion

After median blur + Gaussian blur preprocessing and re-tuned Canny/Hough parameters, the number of detected lines dropped from 1576 spurious segments to 18 accurate lines that correctly correspond to the four real lines in the scene, confirming that preprocessing and correct parameter selection directly determine Hough Transform accuracy in noisy conditions.

Implementation (Python / OpenCV)

Full source code is provided separately as **Q3_hough_transform_noisy_images.py**. The key implementation is reproduced below.

```
"""
Question 3: Hough Transform in Noisy Images
-------------------------------------------
A system uses Hough Transform to detect lines in noisy images but produces
inaccurate results. This script demonstrates the problem and the fix.

Pipeline:
1. Create a clean synthetic image with straight lines.
2. Add Gaussian + salt-and-pepper noise to simulate a noisy sensor image.
3. Run Canny + Hough Transform directly on the noisy image  -&gt; many
   spurious / broken line detections (the problem).
4. Apply preprocessing (median blur to kill salt-and-pepper noise,
   Gaussian blur to smooth remaining noise) and re-tune the Canny /
   Hough parameters -&gt; clean, accurate line detection (the solution).
5. Save a 4-panel comparison figure as the output screenshot.
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt


def build_clean_line_image(size=500):
    """Create a synthetic image containing a few straight lines."""
    img = np.zeros((size, size), dtype=np.uint8)
    cv2.line(img, (50, 50), (450, 50), 255, 3)
    cv2.line(img, (50, 50), (50, 450), 255, 3)
    cv2.line(img, (60, 400), (440, 100), 255, 3)
    cv2.line(img, (100, 450), (450, 450), 255, 3)
    return img


def add_noise(img):
    """Add Gaussian noise + salt-and-pepper noise to simulate a noisy capture."""
    noisy = img.astype(np.float32)

    # Gaussian noise
    gaussian_noise = np.random.normal(0, 35, img.shape).astype(np.float32)
    noisy = noisy + gaussian_noise

    # Salt-and-pepper noise
    salt_pepper_mask = np.random.rand(*img.shape)
    noisy[salt_pepper_mask &lt; 0.02] = 255
    noisy[salt_pepper_mask &gt; 0.98] = 0

    noisy = np.clip(noisy, 0, 255).astype(np.uint8)
    return noisy


def detect_lines_naive(noisy_img):
    """Run Hough Transform directly on the noisy image (no preprocessing)."""
    edges = cv2.Canny(noisy_img, 50, 150)
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=60,
                            minLineLength=30, maxLineGap=5)
    result = cv2.cvtColor(noisy_img, cv2.COLOR_GRAY2BGR)
    n_lines = 0 if lines is None else len(lines)
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line[0]
            cv2.line(result, (x1, y1), (x2, y2), (0, 0, 255), 2)
    return edges, result, n_lines


def detect_lines_improved(noisy_img):
    """Preprocess the noisy image, then run Hough Transform with tuned params."""
    # Median blur removes salt-and-pepper noise very effectively
    denoised = cv2.medianBlur(noisy_img, 5)
    # Gaussian blur smooths remaining Gaussian noise before edge detection
    denoised = cv2.GaussianBlur(denoised, (5, 5), 1.2)

    edges = cv2.Canny(denoised, 60, 160)
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=100,
                            minLineLength=80, maxLineGap=10)
    result = cv2.cvtColor(noisy_img, cv2.COLOR_GRAY2BGR)
    n_lines = 0 if lines is None else len(lines)
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line[0]
            cv2.line(result, (x1, y1), (x2, y2), (0, 255, 0), 2)
    return denoised, edges, result, n_lines
def main():
    np.random.seed(7)
```

```
    clean = build_clean_line_image()
    noisy = add_noise(clean)

    naive_edges, naive_result, n_naive = detect_lines_naive(noisy)
    denoised, imp_edges, imp_result, n_improved = detect_lines_improved(noisy)

    print(f"Lines detected WITHOUT preprocessing: {n_naive}")
    print(f"Lines detected WITH preprocessing:    {n_improved}")

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    axes[0, 0].imshow(clean, cmap='gray')
    axes[0, 0].set_title('1. Clean Synthetic Image')
    axes[0, 0].axis('off')

    axes[0, 1].imshow(noisy, cmap='gray')
    axes[0, 1].set_title('2. Noisy Image\n(Gaussian + Salt-Pepper Noise)')
    axes[0, 1].axis('off')

    axes[0, 2].imshow(naive_edges, cmap='gray')
    axes[0, 2].set_title('3. Canny Edges (No Preprocessing)\nNoisy, fragmented edges')
    axes[0, 2].axis('off')

    axes[1, 0].imshow(cv2.cvtColor(naive_result, cv2.COLOR_BGR2RGB))
    axes[1, 0].set_title(f'4. Hough Result - NO Preprocessing\n{n_naive} spurious lines detected')
    axes[1, 0].axis('off')

    axes[1, 1].imshow(imp_edges, cmap='gray')
    axes[1, 1].set_title('5. Canny Edges (After Median +\nGaussian Blur Preprocessing)')
    axes[1, 1].axis('off')

    axes[1, 2].imshow(cv2.cvtColor(imp_result, cv2.COLOR_BGR2RGB))
    axes[1, 2].set_title(f'6. Hough Result - WITH Preprocessing\n{n_improved} accurate lines detected')
    axes[1, 2].axis('off')

    plt.suptitle('Q3: Effect of Noise on Hough Transform Line Detection', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.savefig('/home/claude/cv_solutions/images/Q3_output.png', dpi=150, bbox_inches='tight')
    print("Saved output screenshot to Q3_output.png")


if __name__ == "__main__":
    main()
```